

なばり情報システム入門

「集める」から「Web に表示する」まで

Hiroki Funashima

このシステムで、何ができる？

名張市役所と名張市観光協会の Web サイトからお知らせを集め、自分のコンピュータに保存する。利用者が「ごみ」「祭り」などの言葉で検索すると、保存した情報から条件に合う記事を選び、Web ページにして返す。

図書委員がお知らせを集めて、出典付きのカードに整理し、質問に合うカードを見せる仕事を思い浮かべよう。このシステムは、その一部をプログラムで行う。人が記事の内容を正しいと判定する仕事まで自動化したわけではない。

読み終わったときの目標

- どこから、何を集めているか説明できる。
- 検索欄と収集設定の違いを説明できる。
- HTML、JSON、SQLite の役割を区別できる。
- 「30 件取得できた」と「使いやすいことが証明できた」の違いが分かる。
- 付録の短い Python コードを、入力・処理・出力に分けて読める。

この本の順番

最初に全体の動きを見て、入力データと設定を確認する。次に実際の結果と、困ったときの見方を学ぶ。付録では、使っているライブラリと実装コードを順に読む。初めはコードを全部暗記しなくてよい。「この部分は何のためにあるか」を自分の言葉で説明してみよう。

本書の画面や件数は 2026 年 9 月 29 日に確認した版に対応する。自動収集を実行すると、その後の件数や記事は変わる。ここに示すテスト結果はソフトウェアの動作確認であり、利用者の実験結果ではない。

1. やっていることを 5 段階に分ける

段階	たとえば	プログラムの仕事
1 取得	掲示板を見に行く	公式サイト の HTML を受信する
2 抽出・整理	題名と必要な部分をメモする	題名・本文冒頭・掲載日等を取り出す
3 保存	整理したカードを棚に入れる	SQLite の表へ保存する
4 検索	条件に合うカードを探す	言葉・出典・カテゴリで絞る
5 表示	見つかったカードを見せる	Flask が HTML を作り、ブラウザへ返す

「取得・整理・保存」は、一般に ETL と呼ぶ考え方に対応する。Extract は取り出す、Transform は形を整える、Load は保存先へ入れる、という意味である。検索と表示は、その保存されたデータを利用する別の処理になる。

いつ動く？

標準の起動方法では、起動直後に収集する。その収集が終わった後、約 30 分待って次の収集を行う。Web ページを開くたびに公式サイト全体を取りに行くわけではない。利用者の検索には、すでに保存したデータを使う。

そのため、検索する人が増えても、同じだけ公式サイトへの通信回数が増えるわけではない。一方、公式サイトが今更新されたとしても、このシステムがまだ取得していなければ画面には反映されない。

「動的な Web ページ」とは？

例えば検索欄を「ごみ」から「祭り」に変えると、Flask が違う条件で DB を調べ、結果に応じた HTML を作る。記事が増えれば、次にページを開いたときに表示内容も変わる。開きっぱなしのページが自動で最新表示に切り替わる機能はなく、再読み込みで更新する。

Flask は Web アプリを作るための道具で、SQLite は情報を保存する道具である。どちらも生成 AI ではない。このシステムの本文は文章生成による要約ではなく、元ページの冒頭を短く抜き出したものである。

実装の正確な動きは同梱コードを、実行時の取得件数は results/collection_runs.json を確認する。説明用の短縮コードだけを本体へ貼り付けず、元の関数とテストも一緒に読もう。研究で支援を受けた範囲を明記し、自分が理解・変更・測定した内容を説明することが大切である。

2. 利用者が入力する項目

Web 画面の入力は「何を表示してほしいか」を指定する。収集元の公式サイトを書き換えたり、記事を投稿したりするものではない。

項目	意味	例・注意
言葉で探す	題名や短い抜粋に含まれる語を探す	「ごみ」「秋 祭り」。100 文字まで
情報の出典	どちらの団体の情報を見るか	両方／名張市役所／名張市観光協会
カテゴリ	内容の種類を指定する	観光・お出かけ、健康・福祉など
ページ	結果を 12 件ずつ見る	次のページ・前のページで移動
文字の大きさ	表示する文字を変える	検索結果の内容は変わらない
色をはっきり	見やすい配色へ変える	端末内に設定が残る

複数の言葉を入れると？

「秋 祭り」は、空白で分けた両方の語を含む記事を探す。これは AND 検索という。「秋だけ」または「祭りだけ」でもよい OR 検索とは違う。出典やカテゴリも選ぶと、さらに条件が増えるため、結果が 0 件になる場合がある。

検索する前に文字を整える。「ゴミ」「ゴミ」「ごみ」のような表記の違いをそろえる処理を正規化という。ただし、意味が似ている語を自動で推測する機能ではない。「清掃」と「ごみ」が同じ意味として必ず一致するわけではない。

カテゴリの一覧

内部の値	画面で見える名前
event / tourism	イベント / 観光・お出かけ
waste / health	ごみ・資源 / 健康・福祉
consult / disaster	相談 / 防災
city	くらし・手続き

内部の source=city は「出典が市役所」、category=city は「分類がくらし・手続き」を表す。同じ city という文字でも、項目名が違えば意味が違う。URL や JSON を読むときは、名前と値を組にして確認しよう。

3. 入力データ：HTML・JSON・SQLite

HTML は、Web ページの構造を書いた文書

ブラウザの画面の裏側には、見出しや段落を指定する HTML という文書がある。次は構造を理解するための架空の例で、実データではない。

```
<article class="article">
  <h1>ごみのお知らせ</h1>
  <p class="right">更新日：2026 年 9 月 20 日</p>
  <div class="txtbox">出し方の説明です。</div>
</article>
```

h1 は大きな見出し、p は段落、div は内容をまとめる箱のようなものだ。class は箱の種類を区別するラベルとして使える。BeautifulSoup はこの構造を読んで、必要な部分を探す道具である。

市役所の記事では article.articleh1 など、観光協会では article.news-posth1.wp-entry-title などを目印にする。公式サイトの作りが違うので、同じ目印を両方に使うと取得できないことがある。掲載形式が変わったときは、プログラムの見直しが必要になる。

JSON は、項目名と値を受け渡し形式

次は説明用に一部だけ取り出した例で、そのまま取込に使える完全なファイルではない。

```
{
  "source_key": "city",
  "title": "ごみのお知らせ",
  "category": "waste",
  "source_date": "2026-09-20",
  "revision": 1
}
```

左の名前と右の値をコロンで結び、項目の間をコンマで区切る。文字列は引用符で囲む。revision の 1 は数なので引用符がない。日付が分からない項目には null が入る場合がある。

SQLite は、情報を表として保存する仕組み

SQLite の DB は、主に 1 つのファイルに表を保存する。JSON は情報を持ち運ぶため、SQLite は繰り返し検索・更新するために使う。HTML・JSON・SQLite は、それぞれ違う役割を担当している。

4. 保存される項目を読む

1 件のお知らせを、次のような項目に分けて保存する。これを「構造化する」と呼ぶ。項目の名前がそろうと、出典が違っていても同じ検索画面で扱える。

項目	意味・読み方
id	記事を識別する短い文字列。URL から作る
source_key	city なら名張市役所、tourism なら名張市観光協会
url	元の公式記事のアドレス。同じ URL の行を増やさない
title	記事の題名
excerpt	本文冒頭の 180 文字まで。続きがあれば「…」を付ける
category	題名の言葉等で自動的に付けた分類
source_date	元ページの更新日または掲載日。開催日ではない
date_kind	updated は更新日、published は掲載日を表す
fetch_at	最後にこのシステムが取得した時刻
first_seen_at	この DB に初めて保存した時刻
changed_at	最後に内容の変化を検出した時刻
revision	保存した内容の版番号。最初は 1
content_hash	前回と同じ内容か比べるための文字列
search_text	検索しやすいよう表記を整えた題名・抜粋等

ハッシュは「内容から作る目印」

SHA-256 という計算で、抽出した本文等から一定の長さの文字列を作る。本文の後半に変更があれば、先頭の抜粋が同じでもハッシュが変わるので、変更を検出できる。ただしハッシュは、記事の正しさや信頼性を採点する点数ではない。

同じ URL・同じハッシュなら、版番号は増やさず取得時刻だけを更新する。異なれば、版を増やし、題名や短い抜粋等の履歴も保存する。記事の全文はハッシュ計算のために解析するが、DB には全文を残さない。

3 種類の日付を混ぜない

9 月 20 日が掲載日、9 月 29 日が取得日、10 月 24 日が開催日という記事もあり得る。9 月 29 日に取得できたからといって、「9 月 29 日に内容を人が確認した」「9 月 29 日に行事を開催する」という意味にはならない。

5. 運用する人が指定する設定

設定は、プログラムを「どこで、どのくらいの間隔で動かすか」を決める。検索欄の入力とは役割が違う。まず既定値で動かし、必要な設定だけ変えよう。

項目	既定値	意味
-host	127.0.0.1	接続を受け付ける場所。既定では自分の PC から利用
-port	8000	Web サーバの入口番号。URL 末尾の:8000 に対応
-interval	1800 秒	収集が終わってから次の収集までの待ち時間。900 秒以上
-limit	12 件	各新着一覧の先頭から選ぶ件数。1～20
-no-collect	指定なし	指定すると Web 表示だけ動かし、自動収集を止める
NABARI_DATABASE	instance 内の DB	DB ファイルを置く場所を変える環境変数

例えば、次のコマンドは 1 時間おきに収集し、各新着一覧の先頭 10 件を対象にする。

```
python run.py --interval 3600 --limit 10
```

このとき、候補数は市役所 10 件＋定番 7 件＋観光協会 10 件で最大 27 件になる。ただし、新着一覧の件数が少ない、URL が重複している、取得に失敗する等の理由で、保存される数は 27 件より少なくなることがある。

手動で 1 回だけ収集する設定

```
python -m flask --app nabari collect --source tourism --limit 8
```

--source はこの収集コマンドで指定する。all は両方、city は市役所、tourism は観光協会を表す。run.py の引数には --source はないので、コマンド名と引数を組で読む。同じ出典への再収集は、前回完了から 15 分以内だと省略される。

6. プログラム内の設定と処理内容

間隔を短くすると確認頻度は上がるが、相手サイトへの通信も増える。件数を増やすと広く集められるが、時間がかかる。短い間隔や最大件数にするほど良い、というものではない。目的と運用条件に合わせて決める。

設定・場所	現在の内容	理由
SOURCES	2 団体の URL と許可ホスト	意図しないサイトを取得しない
evergreen	市役所の定番 7 ページ	新着に載らなくても生活情報を確認する
CATEGORIES	7 種類の表示カテゴリ	検索条件と表示名をそろえる
通信間隔	最低 1 秒、robots 指定があれば優先	連続した大量アクセスを避ける
接続・受信待ち	15 秒・25 秒	返答を無期限に待たない
MAX_BYTES	2MiB = 2,097,152 バイト	極端に大きいページを処理しない
再実行の間隔	最低 900 秒	同じ出典への連続実行を抑える
状態確認の基準	最後の取得から 24 時間	古い取得記録を利用者に知らせる

これらは主に `sources.py` と `collector.py` に書かれている。上限や許可する URL を変える場合は、意味を理解してから担当教員と確認しよう。データ量と通信先が変わると、テストや利用条件も見直す必要がある。

robots.txt とは？

サイトの運営者が自動収集プログラム向けのルールを示すファイルである。取得してはいけない場所などの指定を確認する。`robots.txt` で許可されることと、画像や文章を自由に転載できることは同じではない。このシステムは短い抜粋と出典リンクを示す。

カテゴリは、どう決める？

`classify()` では題名の言葉などを順に調べる。例えば「ごみ」「電池」等が含まれればごみ・資源へ分類する。相談や健康等の判定も行い、観光協会の記事はそれらに当たらなければ観光・お出かけにする。最後に残った市役所の記事はくらし・手続きへ入る。

これは人が決めたルールであり、機械学習の訓練をしているわけではない。誤分類は起こり得る。直す場合は「この題名はこのカテゴリになる」というテストも追加すると、別の記事への影響を確認しやすい。

7. 結果の読み方

2026 年 9 月 29 日の検証では、次の結果になった。設定した対象に対する結果であり、名張市にある情報の全体数ではない。

出典	取得候補	取得できた件数	状態
名張市役所	19	18	一部失敗
名張市観光協会	12	12	完了
合計	31	30	1 件で 502 エラー

市役所の 1 ページで 502 という HTTP エラーが返った。外部サイトや途中の通信経路が正常に返答できなかったとき等に生じる。この番号だけで原因を断定することはできない。失敗したページを「内容が空の記事」として保存せず、他の 30 件を残している。

取得履歴に出てくる項目

名前	意味
discovered	その回に取得候補として選んだ URL 数
fetched	取得・解析して保存処理まで成功した回数
new	初めて保存した記事数
updated	前回と違う内容に更新した記事数
unchanged	前回と同じ内容だった記事数
errors	取得・解析できなかった URL や理由

成功した取得では、fetched は new + updated + unchanged になる。取得回数と記事の総数は別なので、同じ記事を再取得しただけでは保存件数は増えない。updated が 0 でも、正常に確認して unchanged が増えているなら、更新処理が止まったとは限らない。

保存 30 件なのに、画面は 29 件？

観光協会の 1 件に、本日より先の 2026 年 10 月 1 日という掲載日が付いていた。この版は未来の掲載日を持つ記事を、その日が来るまで一覧・詳細・API に出さない。データは消しておらず、収集状況には件数が示される。取得日と出典の日付を比べる例である。

8. 動かして確かめる

Python 3.11 以上を使う。ZIP を展開し、README があるフォルダでターミナルやコマンドプロンプトを開く。以下は一度だけ準備する手順である。

```
python -m venv .venv
```

仮想環境は、この研究で使う追加ライブラリをまとめて置く場所である。次のうち、自分の環境に合う 1 行で有効にする。

```
# macOS / Linux  
source .venv/bin/activate
```

```
REM Windows command prompt  
.venv\Scripts\activate.bat
```

続けて、必要なライブラリを入れ、同梱データで試す。

```
python -m pip install -r requirements.txt  
python -m flask --app nabari import-demo  
python run.py --no-collect
```

ブラウザで <http://127.0.0.1:8000/> を開く。ライブラリのインストールにはネット接続が必要だが、同梱データを使った表示練習には外部サイトへの収集通信は不要である。既存の DB にデータが入っている場合、import-demo は上書きを拒否する。

確認すること

- 出典を観光協会にして「探す」を押す。記事の出典がそろうか確認する。
- 記事を 1 件開き、掲載日と取得日時を読み分ける。
- Ctrl+C でサーバを止める。もう一度同じ起動コマンドを実行して、情報が残るか確かめる。
- 自動収集を試すときは、Ctrl+C で止めてから pythonrun.py で起動し直す。

Web 画面を閉じることと、Python のサーバを止めることは別である。サーバが止まると、ページの新しい要求に答えたり定期収集したりできなくなる。

9. 困ったときと、研究で言えること

見える状態	確認すること
ページを開けない	サーバが動いているか、URL の番号が合っているか
ライブラリが見つからない	仮想環境を有効にしたか、pip を実行したか
検索が 0 件	言葉・カテゴリ・出典の条件を減らす。初回収集状況を見る
保存件数が増えない	unchanged が増えていないか。重複を防ぐ正常な動作かもしれない
skipped と出る	同じ出典が収集中、または 15 分以内に再実行していないか
network_error	接続と取得先の状態を確認する。短時間に連打しない
structure_changed	公式ページの構造が変わった可能性。目印とパーサを確認
古い取得時刻	サーバ停止、取得失敗、新着対象から外れた可能性を確認

エラーが出たら、時刻、行った操作、表示されたメッセージを残す。「動かなかった」だけでなく、「何を入れ、何を期待し、何が起きたか」を書くと、原因を追いやすい。

テスト 58 件が合格した、とは？

用意した入力に対して、期待した出力や状態になったという意味である。例えば「同じ URL を 2 回保存しても記事数が増えない」「観光協会を指定すると市役所の記事が混ざらない」などを確認した。

合格した 58 件の内訳は、Flask・収集・DB が 29 件、予備実験の集計が 9 件、固定した旧 UI が 20 件である。全ブラウザで表示が崩れないことや、すべての利用者に使いやすいことまで証明した数字ではない。

Appendix A. どんなライブラリを使う？

ライブラリは、よく使う処理をまとめた工具箱である。すべてを一から書く代わりに、公開された道具を組み合わせる。何を任せ、どの部分を自分で決めたかを説明できることが大切だ。

名前	このシステムでの役割	たとえ
Flask	URL ごとに処理を選び、応答を作る	窓口の受付
Jinja2	データを HTML のひな型へ入れる	空欄付きの印刷用紙
requests	公式サイトへ HTTP 通信する	取り寄せ担当
BeautifulSoup	HTML の構造から情報を取り出す	ページの見出しを探す係
Waitress	ブラウザとの通信を受け付けて Flask へ渡す	Web サーバの入口
pytest	Python の期待した動作を自動確認する	確認手順を繰り返す係

Jinja2 は Flask の依存として一緒に入る。requirements.txt は主な実行時ライブラリの版を指定する。requirements-dev.txt にはテスト用の pytest も含む。Python で使う関数名と、pip で入れるパッケージ名が違う場合もある。例えば BeautifulSoup は beautifulsoup4 をインストールし、コードでは bs4 から読み込む。

Python にもともと含まれる道具

名前	役割
sqlite3	SQLite へ保存・検索する
hashlib	内容のハッシュを計算する
json	JSON の文字列と Python のデータを変換する
datetime	日付と時刻を扱う
threading	定期収集と Web 応答を同じプロセス内で並行して進める
argparse	-interval などの起動引数を読む
urllib.robotparser	robots.txt の指示を読む

ブラウザ側は JavaScript と CSS を使う。Node.js と jsdom は固定旧 UI のテスト専用で、Flask アプリを動かすためには不要である。本文 PDF は LuaLaTeX で組版する。LuaLaTeX も、情報を収集する Flask サーバとは別の道具である。

Appendix B. コードを入力・処理・出力で読む

B1. HTML の中から題名を探す

実装の考え方を示す短い例である。実際には存在確認や上限の検査も行う。

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(html_bytes, "html.parser")
heading = soup.select_one("article.article h1")
title = heading.get_text(" ", strip=True)
```

入力を受信した HTML、処理は目印に合う要素を探して文字を取り出すこと、出力は `title` という文字列になる。`select_one` は最初に見つかった 1 個を返す。何も見つからなければ `None` なので、実装ではその状態を検出して失敗を記録する。

B2. 表記を整える

```
text = unicodedata.normalize("NFKC", text).casefold()
```

実装の `normalize` 関数にある処理である。NFKC で全角半角などの違いを整え、`casefold` で英字の大小の違いを整える。続くコードでカタカナをひらがなに寄せ、余分な空白もまとめる。入力の意味を理解する処理ではなく、文字の表し方をそろえる処理である。

B3. 保存済みか調べる

```
old = db.execute(
    "SELECT * FROM articles WHERE url=?",
    (article_url,)
).fetchone()
```

SQL は DB へ質問するための言語である。これは「`articles` の表から、この URL の行を探す」という質問だ。? の場所へ別に渡した値を入れる。利用者の文字列を SQL の命令文へ直接つなげない書き方になっている。

`old` がありハッシュも同じなら、取得時刻だけを更新する。なければ新しい行を作る。保存内容が違えば更新し、`revision` を 1 増やす。実際の処理は `nabari/db.py` の `save_article()` を読むと追える。

Appendix C. Web のコードと読む順番

C1. URL から処理を選ぶ

```
@bp.get("/")
def index():
    result = query_articles()
    return render_template("index.html", **result)
```

これは仕組みを示すために短くした例である。実際の `index` 関数は、収集状況やページ送り用の情報もテンプレートへ渡す。`@bp.get("/")` はトップページの要求を、この関数で処理するという指定だ。

`query_articles()` が DB を検索し、結果を辞書にまとめる。`render_template()` は HTML のひな型に結果を入れる。`**result` は辞書の中身を名前付きの引数として渡す書き方である。

C2. テンプレートに題名を入れる

```
<h1>{{ article.title }}</h1>
<p>{{ article.excerpt }}</p>
```

二重の波かっこの位置へデータが入る。Jinja は通常、HTML として特別な意味を持つ文字を安全な表示へ変換する。外部サイトの文章に HTML に似た文字があっても、そのままプログラムとして実行させないための重要な仕組みである。

ファイルを読むおすすめの順番

順番	読むもの	注目する問い
1	README.md	どう起動し、どの URL を見るか
2	sources.py	どのサイトとページを取得するか
3	collector.py	どう取り出し、失敗をどう扱うか
4	db.py	何を保存し、変更をどう見分けるか
5	routes.py / templates	検索条件がどう画面になるか
6	test_flask_system.py	どんな入力と期待結果を確認するか

用語と、次に調べる資料

用語	高校生向けの説明
サーバ	要求を受け取り、結果を返すプログラムやコンピュータ
クライアント	要求を送る側。この場合は主にブラウザ
HTTP	Web の情報をやり取りするための決まり
URL	Web 上の情報の場所を表す文字列
API	プログラム同士が決まった形で情報を受け渡す入口
パーサ	文書の構造を読み取る処理
スナップショット	ある時点のデータを固定して保存したもの
タイムアウト	決めた待ち時間を過ぎたら処理を打ち切ること
コミット	DB の変更を確定して保存すること
テスト用データ	動作を確かめるために用意した入力。実際の住民データとは別

小さな確認課題

「同じ記事を 2 回取得したら、何が変わり、何がかわらないか」を説明してみよう。答えは、記事の内容が同じなら最終取得時刻は変わるが、記事の総数と版番号は増えない、である。新しい機能を加えるときも、このように期待する結果を先に言葉にすると考えやすい。

公式情報と技術資料

名張市役所 新着情報一覧

<https://www.city.nabari.lg.jp/news.html>

名張市観光協会 お知らせ

<https://kankou-nabari.jp/news>

Flask 公式ドキュメント

<https://flask.palletsprojects.com/en/stable/>

Python 標準ライブラリの説明

<https://docs.python.org/ja/3/library/>

LuaTeX-ja (日本語を LuaLaTeX で組版するためのパッケージ)

<https://ctan.org/pkg/luatexja>